Problem 1: Finding Microdrama Promos

Show: I Married a Broken Soldier. He was a Duke (50 episodes)

Live App: https://promobox-p5ztlogtutmnxgvxcky2ea.streamlit.app/ | Code & Outputs: https://github.com/rajvk23/promobox

1. Why I Picked Problem 1 & My Assumptions

Watching 50 episodes by hand every week just to find 20-60 second promo clips is slow, repetitive work. When I opened the input pack, I noticed the screenplays already have exact timestamps (`Start` and `End`) for every line of dialogue, stage action, camera shot, and sound cue, plus emotion tags in parentheses. That meant I did not need an AI to guess timestamps. I could use plain Python code to slice clean candidate clips and save AI for the part rules cannot easily do: judging whether a clip actually hooks a stranger and leaves them hanging.

Where the brief left things open, I made four practical assumptions:

- **One continuous window per promo (20-60s):** The brief allows multi-segment mashups, but jumping across scenes without video editing or music feels jarring. I scoped my tool to single continuous stretches (with 25-50s treated as the sweet spot for Reels/TikTok/Shorts).
- **Never cut a spoken line in half:** Every candidate window must start at the exact beginning of a dialogue or action event and end at the completion of an event. This guarantees every timestamp we hand an editor is a clean cut point.
- **Keep up to 2 non-overlapping clips per episode:** Across 50 episodes this yields 100 promos total, ranked globally so the team knows which 5-10 clips to post first.
- **Always work offline if no API key is set:** So the tool never breaks during a live demo or rate limit, every LLM step has a fast offline fallback.

2. How It Works (What is Code vs. ML vs. LLM)

Dumping 50 full screenplays straight into an LLM is expensive, slow, and prone to made-up timestamps. Instead, I built a 4-stage funnel (`run.py`) that narrows **16,600+ valid windows** down to the best picks:

- **Stage 1 - Parser & Window Proposer (Plain Code, promo_finder.py):** Reads each episode, repairs UTF-8 encoding glitches, extracts dialogue/emotion/action/sound blocks, and proposes every valid 20-60s window (~150 to 650 per episode). Runs across all 50 episodes in under 1 second.
- **Stage 2a - Rule Scorer (Plain Code, promo_finder.py):** Scores every window on four signals: **Hook** (first 3s: physical actions like slaps/strikes, exclamations, high-intensity emotions, while penalising weak openers like 'Yes' or 'And'), **Cliffhanger** (last 6s: episode-end stings, questions, threats, while penalising resolved lines like 'Thank you'), **Standalone** (bonus for starting at a scene boundary, fewer than 4 speakers, and earlier episodes that need less backstory), and **Intensity**.
- **Stage 2b - Self-Supervised Hook & Cliffhanger Model (ML, learn.py):** I realized the 50 episodes already contain free training labels written by professional writers: the first 3s of each episode is a real hook, and the last 6s is a real cliffhanger. I trained a lightweight `scikit-learn` Logistic Regression classifier on 14 show-agnostic structural features (punctuation, emotion intensity, dialogue density) to tell episode edges apart from random mid-episode moments, and blended its probability 50/50 with the rule scores. In 5-fold leave-episode-out cross-validation (`models/cv_report.json`), it reaches **0.789 AUC** on hooks and **0.826 AUC** on cliffhangers.
- **Stage 3 - Cold-Viewer Curiosity-Gap Check (LLM, judge.py):** Rules are bad at telling whether a clip makes sense to someone who has never seen the show. For the top 4 windows per episode (200 total), I pass *only* the clip transcript to Gemini (`gemini-3.1-flash-lite`, swappable via `LLM_MODEL` and `LLM_BASE_URL`) and ask it to act as a first-time viewer: list any plot-breaking `past_confusions`, list urgent `future_questions`, and rate standalone clarity and cliffhanger pull from 0 to 10.
- **Stage 4 - Pairwise Tournament Ranking (LLM, rank.py):** Asking an LLM for isolated 0-10 scores across 50 episodes is noisy. To rank the top 12 finalists, I run a round-robin tournament (66 pairs, asking 'which clip would you post first?'). Because LLMs have position bias, every pair is tested in **both orders** (`A vs B` and `B vs A`, 132 calls total); a clip only gets a win if it wins both directions.

3. Scaling to Every Show Kuku Makes

Stages 1 and 2 run on all 50 episodes in **2.5 seconds on a laptop CPU for $0**. Stage 3 makes 200 short LLM calls and Stage 4 makes 132 calls (332 calls per 50-episode show, ~165k tokens total). Using `gemini-3.1-flash-lite` with parallel threads and disk caching, a full 50-episode show finishes in **~40 seconds and costs ~$0.03** (or $0 on the free tier). Even at 20 new shows a week, the compute bill is under $1/week, and an editor only reviews the top 10-15 ranked clips instead of watching 50 episodes from scratch.

4. Top Ranked Promos: Which Ones I Would Post First & Why

The full ranked list of 100 promos with transcripts is in `outputs/promos.md`. If I were running user acquisition for this show, these are the first five clips I would post:

- **#1 (Post First) - Ep 31, 02:02.00 -> 02:40.00 (38.0s | Score 6.12 | Tournament: 11/11 Wins):** Opens on a physical blow landing on Catherine and Juliana's defiant line ('If you were truly powerful, Catherine, you wouldn't need me to bow'). Escalates as Catherine burns Juliana's late mother's book and holds a glowing branding iron inches from Juliana's eyes. **Why post first:** It went undefeated (11/11) in the pairwise tournament. Anyone scrolling understands captor vs. chained prisoner in 2 seconds with zero backstory, and the cut right before the iron touches her face forces the viewer to watch what happens next.

- **#2 (Post Second) - Ep 24, 00:33.80 -> 01:10.50 (36.7s | Score 6.44 | Tournament: 8/11 Wins):** Opens in 1.3s with a noblewoman striking Juliana across the torso with a riding crop ('You are a maid. Not a physician. Not a lady.'). She forces Juliana onto her knees to scrub a soot-blackened fireplace, then whips the crop into young maid Prudence's face: 'Do you want the same punishment?' **Why post second:** Highest overall score (6.44); instant physical hook and classic Cinderella-style mistreatment.

- **#3 (Post Third) - Ep 1, 01:47.50 -> 02:17.50 (30.0s | Score 6.27 | Tournament: 8/11 Wins):** Opens on Juliana's sister Clarissa gloating in a gold gown at the altar ('You are the daughter people tolerate because they have to. I am the daughter they choose.'). The groom calls Juliana 'low value' in front of the church as Juliana clenches her mother's wedding dress in cold fury. **Why post third:** Sets up the core premise of the entire series for cold audiences without spoiling any later twists.

- **#4 & #5 - Ep 30 (00:08.80 -> 00:39.80, 31.0s) & Ep 50 (00:49.80 -> 01:18.50, 28.7s):** Ep 30 hooks with a conspiracy line ('You have a rat in your cellar, Rupert') and ends on an order to murder Juliana because she knows the Duchess was poisoned. Ep 50 ends on maid Agnes peeking through a cracked door, seeing Lady Catherine in bed with the shirtless priest Father Thomas, and gasping in terror before an abrupt cut to black.

5. Checking Quality & Where My Tool Picks Badly

To check quality objectively (`eval.py`), I first ran automated sanity tests across all 100 output promos: **100/100** stay strictly inside 20-60s and **0/100** cut a dialogue line in half. Second, I built a 24-clip benchmark (`gold/gold.csv`, 8 top, 8 middle, 8 bottom, shuffled blind) and labelled each clip 1 (good promo) or 0 (bad promo). All variants hit **Precision@6 = 1.00**, while ROC-AUC improved from **0.972** (rules only) to **0.979-0.983** when adding the learned model and curiosity-gap score.

More importantly, reading through the 100 outputs showed four clear places where the tool picks badly:

- **1. Tricked by the word 'cliffhanger' in stage directions (Ep 16, 00:41.20 -> 01:23.19, Ranked #7):** This clip starts great (Alexander choking a thug against a wall: 'Who sent you?'), but the thug confesses and runs away at 00:56. The last 25 seconds are just Alexander and Juliana looking into each other's eyes. However, because the writer wrote `cello melody swelling to a dramatic cliffhanger hold` in the final SOUND FX line, my rule scorer gave it a high cliffhanger score even though the scene's tension was already resolved.

- **2. Weak opening rescued by a massive ending (Ep 50, 00:49.80 -> 01:18.50, Ranked #5):** The ending (Agnes catching Catherine and the priest in bed) is a 10/10 scandal cliffhanger, so the clip ranks #5. But the first 3 seconds ('I've told myself that since the dawn in your bedchamber...') is a quiet, slow opener (rule hook = 2.0). A human editor would trim the first 8 seconds or put a 2-second flash of Agnes's gasp at the front.

- **3. Forcing 2 picks on quiet filler episodes (Ranks #85 to #100):** Because `per-episode 2` forces two clips from every episode, quiet transition episodes (like Ep 8 at #100, Score 2.56, where they calmly discuss herbs by the fireplace) produce boring clips. In real use, we should filter by `Score >= 5.5` instead of keeping a fixed number per episode.

- **4. Heavy late-season spoilers (Ep 45 at #12, Ep 42 at #9):** Clips from Episodes 42-45 have high drama ('You poisoned my mother. You tortured my wife.'), but they give away the show's biggest identity reveal and lean on 40 episodes of plot.

6. AI Tools Used & Two Places Where AI Got It Wrong

I used Claude and Gemini inside the Antigravity IDE to help write and debug the Python pipeline and `scikit-learn` training code, and `gemini-3.1-flash-lite` via REST API for the runtime Cold-Viewer and Tournament steps. Two specific things went wrong during development that I had to catch and fix:

- **1. My first ML model cheated on a data confound:** When I first trained the self-supervised classifier (`learn.py`), TF-IDF + Logistic Regression got an 0.85+ AUC that looked great on paper. When I printed its top weights, I saw it was just latching onto the token `ACT` (because episodes almost always start and end on an `ACTION` line, whereas random mid-episode lines are mostly dialogue) and plot words like `'ten days'`. Worse, reading Episodes 6, 7, and 8 showed that ~40% of episode endings in this romance show are warm, peaceful moments by the fire, not cliffhangers! I fixed this by matching negative samples by event type, filtering positive training edges with the rule median, and shipping only the 14 show-agnostic structural features.

- **2. Gemini padded lists in my first Curiosity-Gap prompt:** In my first test of `judge.py` on Episode 1, I asked Gemini

to list `past_confusions` and `future_questions`. Trying to be helpful, the model invented 4 items for every single clip (counting normal curiosity like 'Why is she wearing her mother's dress?' as confusion), which pegged my count formula at 8.0/10 across the board. I rewrote the prompt so `past_confusions` only counts plot-breaking missing context (returning `[]` when a scene is self-explanatory) and combined the counts with explicit 0-10 clarity and pull ratings.